

Diagnóstico Inteligente de Logs

O PROBLEMA

Desenvolvedores e equipes de suporte perdem muito tempo analisando logs longos e complexos de aplicações Java/Spring Boot para identificar a causa raiz de exceções e erros.

OBJETIVO DO AGENTE

Automatizar a análise: ler o arquivo de log, extrair as exceções relevantes e gerar um relatório de diagnóstico estruturado em Markdown.

ENTRADA E SAÍDA

- **Entrada:** arquivo .log informado via CLI.
- **Saída:** relatório Markdown gravado em output/.

FERRAMENTAS REAIS INTEGRADAS

Ferramentas em Python interagem de forma segura com o sistema de arquivos e preparam os dados antes de qualquer chamada ao LLM:

- **Leitura segura:** restrita ao diretório examples/logs/, com bloqueio estrito contra path traversal.
- **Extração via regex:** captura linhas ERROR/WARN e exceções Java típicas, reduzindo o volume de texto enviado ao LLM.
- **Geração de relatórios:** gravação restrita ao diretório output/, com sanitização forçada do nome do arquivo e extensão Markdown.

Arquitetura e Fluxo de Decisão



DECISÕES CONDICIONAIS

- **Validação:** rejeita arquivos inválidos imediatamente, sem tocar ferramentas nem LLM.
- **Classificação:** avalia se o log contém erros; logs limpos pulam o LLM.
- **Diagnóstico:** chamada única ao LLM, apenas com os trechos extraídos.
- **Validação de saída:** revalida a estrutura Pydantic retornada.

MEMÓRIA DE CURTO PRAZO

Um AgentState (TypedDict) transporta o contexto — erros, evidências, categoria e status — entre os nós durante a execução.

SEGURANÇA E GUARDRAILS

- **Validação determinística:** tamanho máximo de 5 MB e extensões permitidas.
- **Path traversal:** bloqueado na leitura e na escrita.
- **Sanitização:** nome do relatório gerado é forçado a formato seguro.

RESILIÊNCIA E FALLBACK

O sistema é projetado para nunca falhar silenciosamente:

- **Aresta condicional:** LLM falhou, chave ausente ou saída inválida redirecionam o fluxo.
- **Fallback determinístico:** gera relatório básico a partir das extrações regex.